Advanced Engineering Informatics 73 (2026) 104560

Contents lists available at ScienceDirect

Advanced Engineering Informatics

journal homepage: www.elsevier.com/locate/aei

Full length article

Domain-specific SQL generation with LLMs: A hybrid framework combining knowledge graphs and retrieval-augmentation

Ifeoluwa Awotunde [a], Dharmendra Reddy Chitte [b], Yongwei Shan [a,*], Weihua Sheng [c], Hossein Khaleghian [d]

[a] School of Civil and Environmental Engineering, Oklahoma State University, Stillwater, 74078, OK, USA
[b] Computer Science Department, Oklahoma State University, Stillwater, 74078, OK, USA
[c] School of Electrical and Computer Engineering, Oklahoma State University, Stillwater, 74078, OK, USA
[d] InfraTie Solutions, LLC., Stillwater, 74074, OK, USA

ARTICLE INFO

Keywords:
Large language model
Domain knowledge
Sewer asset management
Knowledge graph
Graph retrieval augmented generation

ABSTRACT

Municipal sewer asset management personnel without SQL expertise often rely on additional IT support to retrieve necessary information from their organization's database, which hinders timely decision-making. This study presents a schema-guided RAG, a hybrid text-to-SQL framework that integrates structured knowledge graphs (KGs) for explicit join-path and relational reasoning with retrieval-augmented generation (RAG) for contextual grounding. Designed for domain-specific and multi-relational databases, the schema-guided RAG enables large language models (LLMs) to reason over complex relational structures while mitigating data scarcity through semantic retrieval. The framework was evaluated using both proprietary and open-source LLMs and applied in the sewer asset management domain. Results show consistent improvements in execution accuracy, logical form accuracy, and exact match, with particularly strong gains for queries requiring multi-table joins and nested logic. The schema-guided RAG offers an interpretable approach to natural language queries, supporting efficient, accurate, and explainable access to infrastructure data.

1. Introduction

The wastewater infrastructure in the United States is in critical condition, with many systems operating beyond their intended service life. The American Society of Civil Engineers (ASCE) assigned a D+ grade to the nation's wastewater infrastructure in its 2025 Report Card, highlighting issues such as pipe deterioration, insufficient capacity, and frequent sanitary sewer overflows (SSOs) [1]. The Environmental Protection Agency (EPA) estimates 23,000 to 75,000 SSOs will occur annually, posing significant risks to public health and the environment [2]. These challenges underscore the need for a data-driven asset management approach, which enables monitoring, informed decision-making, optimized maintenance, and extended asset lifespans.

Municipalities have increasingly adopted technologies such as closed-circuit television (CCTV), sonar, and the Sewer Line Rapid Assessment Tool (SL-RAT) to collect condition data, while also maintaining Geographic Information Systems (GIS) and work order records. However, as the volume and complexity of data grow, extracting actionable insights becomes more difficult, particularly for decision-makers without expertise in Structured Query Language (SQL) or database management. Generating correct queries often requires joining multiple tables and navigating complex relational structures, creating a growing demand for tools that allow non-SQL experts to access and analyze database information directly. This naturally motivates the development of domain-specific text-to-SQL solutions such as the schema-guided RAG framework presented in this study.

These challenges, especially the need for complex multi-table joins, highlight the complexities involved in automating SQL generation for professional domains. Queries in such settings often reference multiple tables, many professional terms, similar entity names, and entity semantic relationships. The advent of Transformer-based large language models (LLMs) by Vaswani et al. [3] has served as a powerful tool for text understanding and generation, and it represents a promising solution for democratizing access to relational databases. However, out-of-the-box LLMs often struggle with domain-specific SQL generation, particularly in settings with complex multi-table schemas. These challenges stem from their lack of deep understanding of relational schema semantics, such as foreign keys, join paths, and hierarchical relationships.

While fine-tuned models can partially address these limitations, many municipal systems operate in resource-constrained settings where

* Corresponding author.
E-mail address: yongwei.shan@okstate.edu (Y. Shan).

https://doi.org/10.1016/j.aei.2026.104560
Received 7 September 2025; Received in revised form 22 January 2026; Accepted 14 February 2026
Available online 9 March 2026
1474-0346/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

labeled data is scarce and technical expertise is limited. Furthermore, current LLM approaches typically rely on textual context alone, lacking access to structured schema representations or prior query examples that could aid in complex reasoning. As a result, there is a need for a framework that supports both schema-level understanding and retrieval of contextual knowledge to improve accuracy, interpretability, and usability for domain professionals.

To address the challenges of generating SQL queries from complex, multi-relational databases, we present a schema-guided RAG, a hybrid framework that integrates structured knowledge with RAG to guide LLMs. Unlike existing approaches that combine RAG and knowledge graphs, the schema-guided RAG explicitly combines schema-level reasoning via knowledge graphs with contextual retrieval of relevant query examples to improve multi-table SQL generation. The main contributions of this paper are summarized as follows:

1. We design a schema-guided RAG framework, which systematically integrates schema knowledge from knowledge graphs with retrieval-augmented context to enhance SQL generation accuracy.
2. We develop a sewer asset management knowledge graph that encodes schema-level semantics such as entity types, foreign keys, and join paths to support interpretable SQL generation.
3. We demonstrate the schema-guided RAG's generalizability across diverse LLMs, achieving significant improvements in semantic parsing performance in resource-constrained settings through the combination of structured knowledge and contextual retrieval.

The remainder of this paper is structured as follows: Section 2 reviews related work in text-to-SQL generation and knowledge-enhanced language models. Section 3 describes the schema-guided RAG framework, including LLM finetuning, knowledge graph construction, retrieval mechanisms, and prompting strategies. Section 4 presents the experimental setup and evaluation results. Section 5 discusses the findings, limitations, and opportunities for future work. Finally, Section 6 concludes the paper.

2. Related studies

2.1. Evolution of text-to-SQL methodologies

Early text-to-SQL systems employed rule-based and heuristic techniques, using handcrafted grammars and syntactic templates to convert natural language into structured SQL queries [4–7]. These methods relied on keyword matching, template filling, and syntactic parsing to generate valid SQL. While effective in constrained domains, rule-based approaches are limited in scalability, brittle in the face of linguistic variation, and require significant manual effort to adapt to evolving or complex database schemas.

The emergence of deep learning introduced a data-driven approach, enabling models to learn direct mappings from natural language questions to SQL queries without the need for manually defined rules. Early neural approaches utilized sequence-to-sequence architectures with recurrent models, such as Long Short-Term Memory (LSTM) networks [8] and Gated Recurrent Units (GRUs) [9]. A notable advancement was Seq2 SQL by Zhong et al. [10], which incorporated reinforcement learning to refine SQL generation based on execution outcomes. Building on this, SQLNet [11] introduced sketch-based decoding, reducing syntactic errors by predicting SQL clause structures independently.

However, RNN-based models struggled with modeling long-range dependencies and complex schema relationships, especially in multi-table environments. The introduction of the Transformer architecture [3], with its self-attention mechanism, addressed these challenges by capturing global context in both natural language inputs and schema elements.

Subsequent research leveraged graph-based representations to better encode database schemas. For instance, RAT-SQL [12] employed relational graph attention networks to model interactions between schema components and question tokens, achieving state-of-the-art performance on the Spider benchmark. Another advancement, PICARD [13], augmented decoding by constraining beam search to syntactically valid SQL queries, significantly boosting the accuracy of fine-tuned T5-based models.

Despite these improvements, deep learning-based systems typically require large annotated datasets for training and often generalize poorly to domain-specific or unseen schemas [14].

More recently, large pretrained language models (LLMs), such as GPT-4, Codex, and LLaMA, have been applied to text-to-SQL tasks with promising results [15–17]. These models demonstrate strong generalization and zero-shot capabilities, making them attractive for practical deployment. However, their performance is highly sensitive to prompt formulation, and they often hallucinate SQL statements when schema context is ambiguous.

To mitigate these issues, recent studies have explored various strategies, including prompt engineering, in-context learning, fine-tuning, and the incorporation of external knowledge. For example, DIN-SQL [18] decomposes complex queries into subtasks, employs schema linking, and incorporates self- correction mechanisms to improve accuracy. DAIL-SQL [19] presents a dual investigation of prompt design and supervised fine-tuning, proposing a demonstration selection strategy based on SQL operator composition. It emphasizes token efficiency and shows that fine-tuned open-source LLMs can perform competitively with proprietary models. Similarly, PURPLE [20] enhances LLM performance by modeling SQL operator composition via a four-level automaton for selecting in-context examples. Its pipeline includes schema pruning, skeleton prediction, and a database adaptation module to reduce hallucination and stabilize outputs.

Although LLMs have substantially advanced text-to-SQL capabilities, their limitations in domain adaptation and schema reasoning persist. This has led to increasing interest in hybrid approaches that integrate structured knowledge and contextual augmentation through Retrieval-Augmented Generation (RAG), enabling more robust performance in specialized domains such as infrastructure asset management.

2.2. Domain- specific applications of large language models with retrieval-augmented generation

The implementation of LLMs in professional domains has been limited due to the scarcity of annotated training data and the complexity of domain-specific relational schemas. These limitations hinder the direct application of fine-tuned models or in-context learning, especially when natural language expressions and SQL structures vary across sectors. To address this challenge, Retrieval-Augmented Generation (RAG) has emerged as a promising approach [21]. RAG architectures enhance language models by incorporating external knowledge through retrieval mechanisms, allowing the model to condition responses on domain-relevant information during inference. By leveraging retrieved documents or structured data, RAG allows LLMs to generate outputs that are both contextually accurate and semantically consistent with the domain.

Recent studies have demonstrated the potential of RAG in various technical and industrial applications. In industrial automation, Koziolek et al. [22] apply RAG to control code generation by retrieving relevant legacy code for improved PLC programming. Similarly, Heredia Álvaro et al. [23] enhance manufacturing quality control using RAG to retrieve product defect histories and contextualize responses. In healthcare, Park et al. [24] evaluate RAG-based phenotyping and highlight the importance of retrieval quality for clinical accuracy.

For environmental and climate applications, Xie et al. [25] design a multi-agent RAG system for climate resilience, demonstrating how modular retrieval supports complex, interdisciplinary reasoning.

In finance, Gondhalekar et al. [26] introduce MultiFinRAG, an approach that integrates text and tables to achieve more accurate financial question answering (QA). For software vulnerability analysis, Xiao et al. [27] propose Hybrid-NL2SVA, which combines RAG with fine-tuning to improve precision.

In education, Gopi et al. [28] apply RAG to generate adaptive quizzes aligned with engineering curricula, showing its value in dynamic content generation. Collectively, these studies validate RAG's effectiveness in aligning LLMs with domain-specific data across sectors.

2.3. Integrating knowledge graphs with retrieval- augmented generation for LLMs

While Retrieval-Augmented Generation (RAG) helps LLMs ground their outputs in contextual documents, its effectiveness is often limited in scenarios that require structured schema reasoning, such as those involving relational databases. To address this, researchers have explored integrating Knowledge Graphs (KGs) into RAG pipelines, enabling LLMs to leverage structured semantic representations alongside unstructured text.

Several recent efforts, particularly in QA and summarization, have proposed variations of KG-enhanced RAG. For instance, GraphRAG, introduced by Edge et al. [29], expands the retrieval context by navigating both local and global semantic neighborhoods of a KG. This approach improves query-focused summarization by enriching the LLM prompt with topologically related nodes and relations. Similarly, Yang et al. [30] present the Pseudo-Knowledge Graph, which guides retrieval using meta-paths and in-graph textual content to provide domain-specific context, particularly for open-domain QA tasks.

Other implementations also focus on domain-specific QA systems. For example, Li et al. [31] enhance LLM-based QA in specialized domains by aligning retrieved facts with structured entities in a domain KG. Zheng et al. [32] apply KG-enhanced LLMs for construction contract review, where legal entities and clauses are modeled as graph nodes to support accurate clause extraction. Guo et al. [33] use KG-based retrieval to standardize and quantify defect-related product knowledge for industrial diagnostics, while Ma et al. [34] apply KG-enhanced RAG to schema matching across heterogeneous databases.

Despite their strengths, most existing KG-enhanced RAG frameworks focus on open-ended generation tasks, such as QA, summarization, and entity linking. These tasks often rely on factual grounding, but not the precise compositional reasoning required for text-to-SQL generation. Although some existing KG-enhanced RAG frameworks have been applied to text-to-SQL generation [35], they typically rely on similarity-based retrieval of prior SQL queries or table embeddings. These approaches often treat tables and columns as loosely connected nodes, using vector similarity or cached examples to suggest joins rather than explicitly modeling the database's underlying relational structure. This can lead to errors in multi-table reasoning, join path selection, and the handling of complex or domain-specific schemas.

In contrast, our schema-guided RAG framework targets domain-specific semantic parsing, where understanding relational schemas is essential. The framework encodes foreign key relationships, join paths, and entity hierarchies within a domain-level knowledge graph tailored to sewer asset management. Each table, column, and relationship is represented as an entity-relation-attribute triple, enabling reasoning over the database structure rather than relying on implicit similarity. Relevant triples are retrieved from the KG and used to construct join-path suggestions, which are then incorporated into the LLM prompt to guide relational reasoning. Example questions and similar tables and columns are retrieved separately via RAG to contextualize the prompt further.

By integrating these sources, the schema-guided RAG allows the LLM to generate SQL that respects multi-table dependencies, foreign-key relationships, and schema hierarchies. Unlike prior frameworks that rely on similarity-based retrieval or exemplar queries, our approach provides a structured, semantically grounded mechanism for navigating complex, domain-specific databases, substantially improving the accuracy of the generated SQL code.

3. Methodology

The proposed framework in this study can be framed as illustrated in Fig. 1 as a schema-guided RAG system, structured around pre-processing, finetuning, retrieval augmented generation, and knowledge graph context provision for the prompt, followed by automated generation of SQL queries to accurately query information from custom databases.

While the RAG and KG components operate in parallel in terms of their respective retrieval mechanisms, the RAG module provides vector-based example and schema context, and the KG module provides explicit join-path guidance; their outputs are integrated into a unified prompt for the language model. This design allows KG information to inform the LLM's reasoning alongside RAG-retrieved content, providing structured relational cues without requiring direct coupling at retrieval.

Thus, the system represents a schema-guided RAG framework that emphasizes explicit schema-level reasoning and structured guidance. The following sections describe the implementation details and the process by which SQL queries are generated from this integrated schema and example context.

3.1. Pre- processing

3.1.1. Query pre- processing

Professional domains have various technical terms whose abbreviations, well-known to experts, are also used in database schemas for brevity and consistency. However, in everyday communication, people often use these terms in their full or varied natural language forms, even though they are typically not included in standard dictionaries. This discrepancy poses additional challenges for mapping user queries to the database schema during automated query understanding.

To address this problem, a pre-processing step is introduced to map natural language expressions to their corresponding data base-specific abbreviations. This entails entity tokenization, applied to the input text through a rule-based matching approach. The process detects and substitutes recognized terms with their standardized forms, ensuring the query aligns with the database schema before additional processing.

3.1.2. Metadata pre- processing

The database metadata is one of the documents provided to both the retrieval-augmented generation (RAG) and the knowledge graph (KG) frameworks for downstream processing. This metadata includes information such as table and column names, as well as primary and foreign keys, all of which are vital for precise queries and schema alignment.

To improve retrieval effectiveness, the metadata is preprocessed before being utilized in the automated system. The process involves further description of table and column names, as well as structuring the metadata into organized formats that support efficient indexing and querying. For instance, table and column names are transformed into a uniform style, and their descriptions are incorporated to deepen the context offered to the models.

The organized metadata allows the RAG to fetch pertinent schema context for natural language queries and enables the KG to more effectively connect query entities and relationships to the database schema.

3.2. Procedure for fine- tuning large language models

A pre-trained transformer-based LLM is fine-tuned to adapt its attention layers to the annotated dataset. For commercially available models, this fine-tuning is often conducted online. For instance, OpenAI offers APIs enabling users to tailor models with their own data. Conversely, various techniques have been devised for fine-tuning locally hosted LLMs, including LoRA, QLoRA, and P-Tuning [36–38].

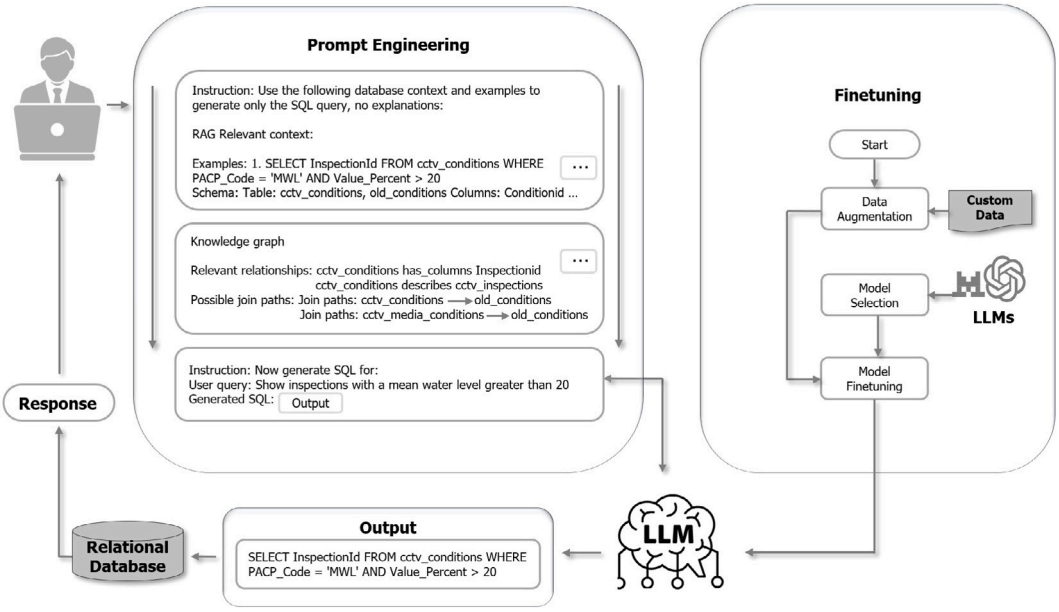


Fig. 1. The overall design of the automated query generation system.

These methods vary in their approach to altering the model's weights and memory usage. LoRA and QLoRA employ low-rank adapters to minimize the computational resources needed for training. In contrast, P-Tuning refines prompt embeddings to guide model behavior without modifying the core network parameters. Considering the requirements for a simple setup, moderate resource usage, and robust performance, LoRA is chosen for fine-tuning locally hosted LLMs in this study.

For commercially available models, the fine-tuning process entails uploading training and validation data to the service, where the provider's infrastructure handles the fine-tuning. This method eliminates the need for local computational resources or model maintenance, but it restricts control over training specifics and model internals. Once fine-tuned, the adjusted model can be utilized for inference on custom tasks through API calls [39]. For locally hosted models, LoRA adapters are first incorporated into the transformer architecture. Then, the annotated dataset is tokenized and structured to align with the model's input specifications. During training, only the LoRA parameters are adjusted, while the base model weights remain constant, thereby reducing memory consumption and accelerating the process. The fine-tuning employs a supervised objective, usually cross-entropy loss, optimized across multiple epochs with hyperparameters determined via validation [36].

3.3. Prompt enhancement via RAG context injection

Retrieval-Augmented Generation (RAG) is a method developed to mitigate the tendency of Large Language Models to hallucinate during response generation due to their limited ability to extend their knowledge base beyond the data on which they are trained [21,40]. The original RAG is an all-inclusive model that utilizes the Dense Passage Retriever (DPR) as the retriever and BART as the generator, which generates an output using the retrieved documents [21]. For the RAG system, professional domain knowledge is stored in a vector database, designed to store text chunks iteratively extracted from the documents in the corpus [29,40].

In this study, a vector database is utilized to store embedded representations of text-to-SQL examples and database metadata, facilitating efficient retrieval driven by semantic similarity. The flowchart (Fig. 2) illustrates the creation of the vector database and the associated retrieval process, which identifies and prioritizes relevant examples and metadata for the augmentation of prompts.

3.3.1. Vector embedding and index construction

To store text chunks in the vector database, each chunk is first transformed into an embedded representation. In this study, embeddings are generated using OpenAI's text-embedding-3-small model, which converts text inputs into high-dimensional vectors capturing their semantic meaning. The embeddings are stored in the database, and the complete set of embeddings is organized into an index structure optimized for similarity search [23].

In this study, the Chroma vector database was selected due to its ease of use and suitability for the lightweight nature of the corpus to be stored. Its ability to perform efficient similarity searches combined with metadata filtering made it a preferred choice over alternatives such as FAISS, Elasticsearch, and Pinecone [41].

3.3.2. RAG context retrieval

The traditional Retrieval-Augmented Generation framework, introduced by Lewis and Perez [21], advocates for single retrieval, where all relevant information is gathered in one step before querying the model for responses. However, several studies have suggested advanced retrieval methods to address issues such as the less effective sequencing of information retrieval and the limitations of single retrieval in handling complex problems that require multi-step reasoning [42]. For this study, however, the issues associated with single retrieval do not apply, leading us to proceed with its implementation.

Given that the dataset and query complexity in this study align well with a single retrieval approach, the developed chroma vector stored embeddings, along with their metadata, allow for precise filtering of chunks during retrieval [42]. In this study, to retrieve relevant context for each system query, the same OpenAI text-embedding-3-small model used to create the vector database is applied to embed the query text, ensuring consistency. The embedded query is then compared to the stored vectors using cosine similarity, as shown in Eq. (1), to identify the most relevant context information, which is subsequently used to enrich the prompt for generating responses.

$$\mathrm{sim}(q, d_i) = \frac{e_q \cdot e_i}{\|e_q\| \cdot \|e_i\|} \tag{1}$$

3.4. Prompt enhancement via knowledge graph context injection

A knowledge graph is a structured knowledge base that organizes entities and their interrelationships, empowering systems to perform

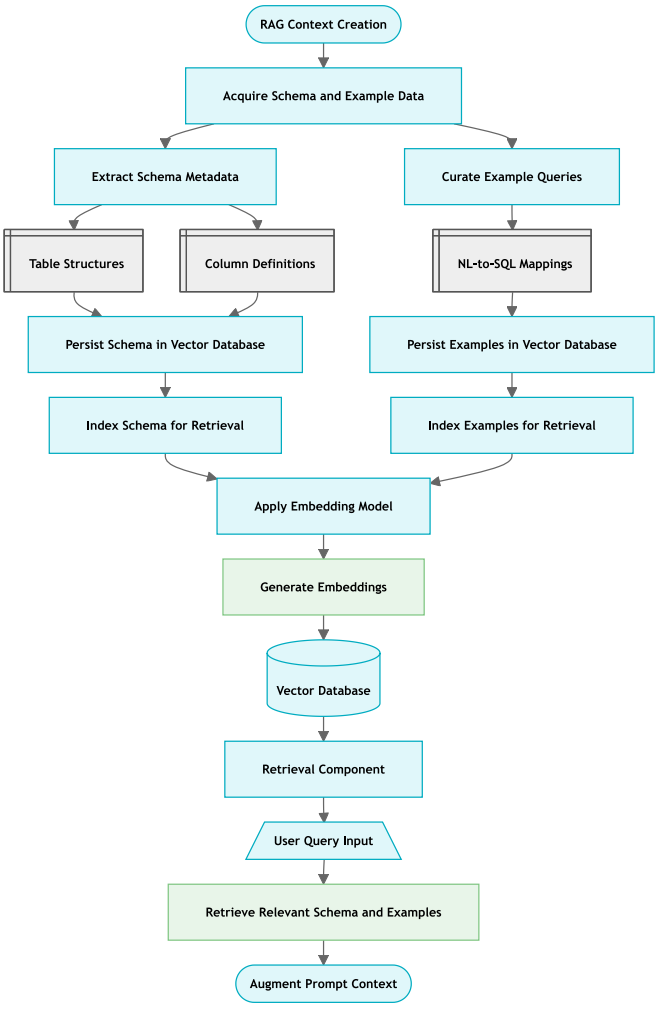


Fig. 2. Retrieval augmented generation pipeline for SQL query generation.

advanced reasoning and derive insights more effectively than from unstructured text alone [43]. The advanced reasoning capability of knowledge graphs has led to their adoption in various professional domains for tasks such as decision support, solution recommendation, and question answering [33,44,45].

In the context of this study, a lightweight domain-specific knowledge graph is used to augment retrieved text with structured information relevant to each query. The flowchart (Fig. 3) illustrates the construction of the knowledge graph and the transformation of the retrieved triples into context suitable for prompt enhancement.

3.4.1. Knowledge graph construction

The knowledge graph in this study is built using a hybrid approach that integrates bottom-up ontology construction with knowledge extraction. Bottom-up ontology construction involves two key components: the data layer and the schema layer. Bottom-up ontology construction begins with the data layer, where entities and their relationships are structured and stored, followed by the schema layer, which defines ontologies, rules, and attribute specifications within the domain of the knowledge graph [33,44].

For the data layer, knowledge extraction techniques were utilized. This included named entity recognition (NER) to identify tables and columns within the metadata, relation extraction (RE) to uncover connections between entities, and entity alignment to consolidate the extracted data into triples [33,45]. These triples serve as the foundation for constructing the knowledge graph, which is then stored in a graph database.

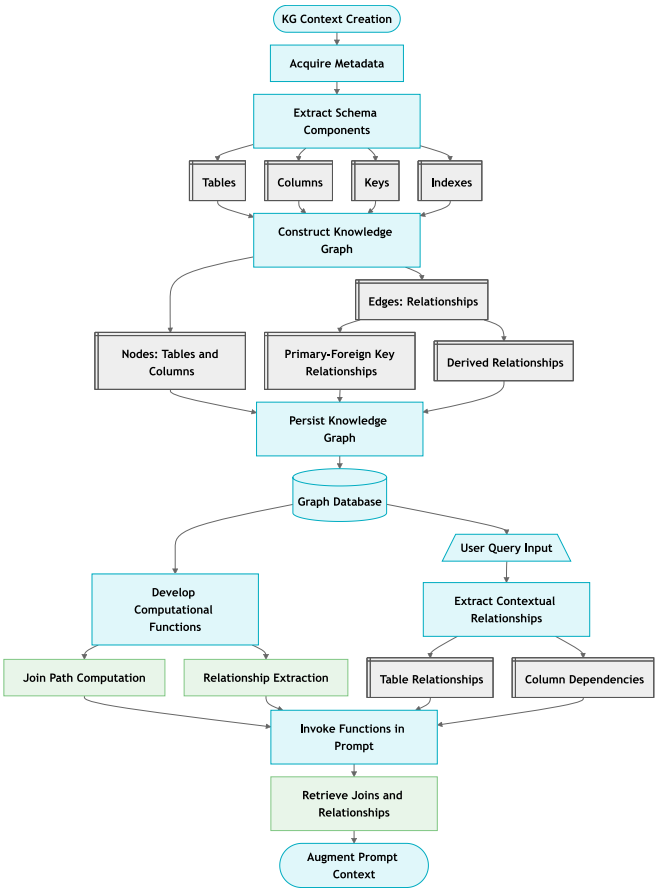


Fig. 3. Knowledge graph pipeline for SQL query generation.

3.4.2. Knowledge graph context retrieval

The knowledge graph retrieval process involves identifying semantically relevant triples in the form of (subject-predicate-object) based on entities extracted from the user query. Eq. (2) defines the query-specific subgraph R_q, which comprises all triples (t,r,n) connected to the query entities. This includes both outgoing and incoming relationships, ensuring that the immediate semantic neighborhood of each entity is captured.

$$R_q = \left\{ (t, r, n) \,\middle|\, t \in T_q, (t, n) \in E \text{ or } (n, t) \in E \right\} \tag{2}$$

Where:

- $t \in T_q$: t is an element in the set of query entities T_q,
- r: The predicate (relation) associated with the triple,
- n: The object (node) connected to t,
- $(t, n) \in E$ or $(n, t) \in E$: The pair (t, n) or (n, t) exists as an edge in the knowledge graph E, meaning there is either an outgoing or incoming relationship between t and n.

To control the size of the retrieved context, a parameter k_{kg} is introduced to constrain the number of knowledge graph triples incorporated into the LLM prompt. The relevance of each triple is assessed based on its semantic proximity to the entities identified in the user query. The retrieved triples are then ranked according to their relevance scores, and only the top-k_{kg} triples are retained for prompt construction.

Rather than assuming a fixed optimal value a priori, k_{kg} is treated as a controllable design parameter whose impact on downstream SQL generation is systematically examined through sensitivity analysis. This approach allows us to assess the trade-off between contextual completeness and noise introduced by less relevant graph entries. By limiting the

prompt to the most salient and semantically aligned triples, the proposed strategy reduces contextual noise while preserving meaningful relational structure for downstream reasoning.

Following a retrieval approach similar to the path query mechanism introduced by Guo et al. [33], the complete set of relevant edges E serves as the foundation for computing expressed in Eq. (3) and recommending potential join paths among the retrieved nodes, typically representing tables and columns in this study's metadata context. These join paths offer a structural representation of how entities are interlinked and can be leveraged to surface composite relationships and enhance schema understanding for prompt enrichment.

The resulting subgraph is then converted into a natural language format for seamless integration into the prompt.

$$P_q = \left\{ \text{ShortestPath}(t_i, t_j) \,\middle|\, t_i, t_j \in T_q, i < j \right\} \tag{3}$$

Where:

- $t_i, t_j \in T_q$: t_i and t_j are nodes (tables or columns) from the set of query entities T_q,
- ShortestPath(t_i, t_j): The shortest path between the nodes t_i and t_j, representing the possible join paths between them,
- $i < j$: The indices i and j satisfy the condition $i < j$, ensuring that each pair of nodes is considered in a unique direction.

In practice, multiple join paths may exist for entities in the user query. To manage this, a parameter k_{join} is defined to control the number of join paths supplied to the LLM prompt as contextual guidance. After computing all shortest paths, the top-k_{join} paths are selected based on path relational relevance. Similar to k_{kg}, the optimal value of k_{join} is determined through hyperparameter tuning to balance contextual completeness and noise reduction in the prompt. These top-ranked join sequences are formatted into human-readable join suggestions that provide the LLM with structured, semantically coherent join-path context, thereby enhancing multi-table reasoning and SQL generation accuracy.

3.5. SQL code generation via LLMs with schema-guided RAG context- enhanced prompts

3.5.1. Schema-guided RAG context enhancement for prompt construction

To improve the accuracy and relevance of generated SQL queries, this study employs a domain-aware context augmentation strategy that utilizes a schema-guided RAG-based prompt enhancement approach. This approach enriches the prompt with SQL-natural language (SQL-NL) examples and structured knowledge extracted from the schema, enabling LLMs to more effectively capture query intent and database structure.

$$\text{Prompt}(q) = \text{Concat}\left[C_1, C_2, C_3, C_4, C_5\right] \tag{4}$$

where:

$C_1 =$ "SchemaContext:" S_q

$C_2 =$ "KGRelationships:" R_q

$C_3 =$ "PossibleJoinPaths:" P_q

$C_4 =$ "Examples:" E_q

$C_5 =$ "NowgenerateSQLfor : Q:" q

The prompt construction strategy is formally expressed in Eq. (4), where the final prompt passed to the LLM is a concatenation of five contextual components, formally defined as follows:

- S_q – **Schema Context**: provides relevant metadata, including table and column names, and description, relevant to the query.
- R_q – **KG Relationships**: includes relevant triples from the knowledge graph, defined in Eq. (2), that describe how tables and columns are semantically related.
- P_q – **Possible Join Paths**: suggests candidate paths, defined in Eq. (3), that logically connect entities from R_q.
- E_q – **Examples**: supplies few-shot demonstrations of SQL queries on similar questions, helping guide the model's formatting and logic.
- q – **User Query**: represents the natural language input for which SQL needs to be generated.

3.5.2. SQL code generation using finetuned large language models

As illustrated in Fig. 4, once the prompt is constructed with schema-guided RAG contextual enhancements as described in Eq. (4), it is passed to an LLM fine-tuned on a SQL-natural language dataset. The fine-tuning process, detailed in Section 3.2, enables the model to learn structural patterns and semantic alignments between natural language inputs and executable SQL statements.

This method helps mitigate common failure modes observed in zero-shot or naïve prompting strategies, such as hallucinated columns, incomplete or incorrect joins, and invalid table references. The inclusion of a few-shot examples further reinforces the model's understanding by demonstrating expected query structures and logic.

To evaluate the effectiveness of this approach, generated queries are tested for execution correctness, logical form accuracy, and lexical alignment with the original question intent. Comparative baselines using only query-informed prompts are included to quantify the benefit of schema-guided RAG context injection.

3.6. Evaluation metrics

To assess the performance of the proposed model, this study adopts three commonly used metrics for evaluating text-to-SQL models, as discussed by Kanburoğlu and Tek [46]. These metrics are Exact Match (EM), Execution Accuracy (EX), and Logical Form (LF), each evaluating a different aspect of query correctness. Execution Accuracy (EX) focuses on the results produced by the generated query, comparing the query's output to the ground truth by executing both on the same database. Logical Form (LF), by contrast, assesses how well the generated SQL query captures the logical structure of the intended query, penalizing cases where the query produces the correct results but lacks exact string matching. Finally, Exact Match (EM) evaluates the generated query by comparing it to the ground truth query, checking for the presence of the same SQL components. Together, these metrics provide a comprehensive evaluation of the model's ability to generate accurate and logically consistent SQL queries.

4. Case study

4.1. Data collection and processing

In this section, the effectiveness of the proposed framework in improving the accuracy of LLMs on domain-specific queries is evaluated. To this end, a case study was conducted using a sewer asset management database, and a curated set of representative questions commonly posed by engineers and other stakeholders to the database. The database features 51 tables and 1114 columns, with several instances of overlapping or similarly named tables and columns. This complexity introduces significant ambiguity, making it challenging for large language models (LLMs) to accurately interpret and generate SQL queries without additional contextual support.

The database schema was preprocessed by standardizing all table and column names to lowercase to ensure consistency. Descriptive metadata for each table and column was also incorporated to enrich the schema with semantic information. Furthermore, tables irrelevant to the core use case were excluded from the schema. These included tables responsible for logging user access, tracking database changes, and handling interactions with the web interface. After preprocessing,

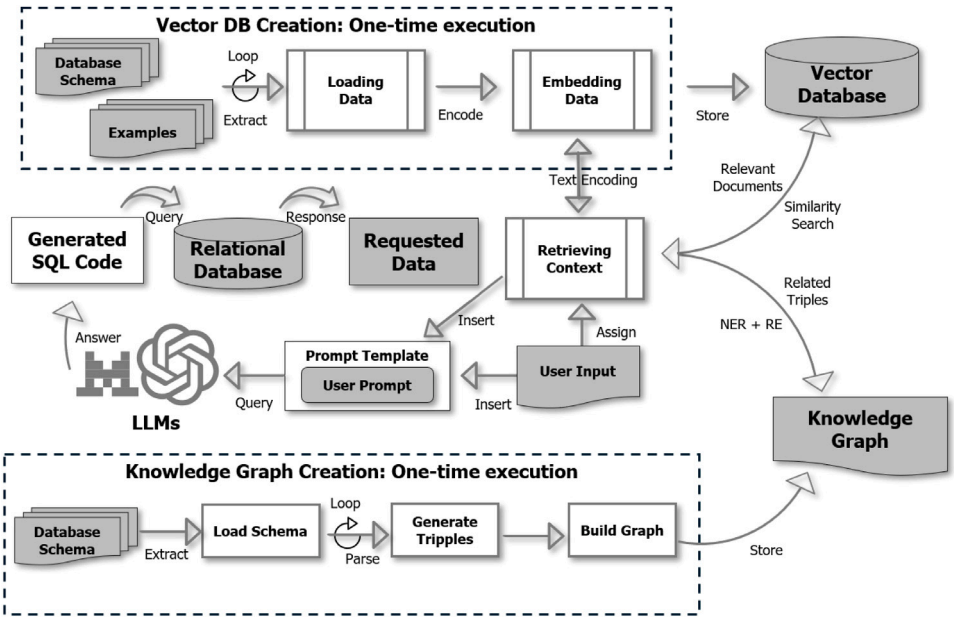


Fig. 4. Design of the implemented schema-guided RAG framework.

Table 1
Retrieved document (top_k) tuning for RAG.

Top_k	EX	Precision@k	Recall@k	F1 score
1	84.38%	90.07%	89.73%	89.90%
3	**90.62%**	**90.64%**	**92.19%**	**91.42%**
5	87.50%	88.02%	89.06%	88.43%
7	87.50%	90.10%	92.19%	91.13%
10	87.50%	90.10%	92.19%	91.13%

the refined metadata consisted of 45 tables and 1083 columns, resulting in a simplified schema that retained all domain-relevant information while reducing noise and potential ambiguity. This preprocessing step was crucial in minimizing the search space and enhancing the precision of context-aware retrieval mechanisms, such as those employed in the schema-guided RAG framework.

A corpus of 55 natural language questions was curated from domain practitioners, encompassing simple single-table queries to complex queries involving nested subqueries, multi-table joins, aggregation, and filtering. Data augmentation techniques, including paraphrasing, synonym substitution, and controlled noise injection, were employed to expand the dataset. Seven variations were generated for simple questions, and fourteen for complex queries, yielding a total of 464 training examples. The dataset was partitioned into training, validation, and test sets following an 80:10:10 ratio. This augmented dataset enhanced the robustness and diversity of the training data, enabling the model to generalize across varied query patterns and ensuring evaluation metrics reflected the complexity and variability inherent in real-world sewer asset management queries.

4.2. Construction of the automated query generation system

The retrieval systems for metadata and few-shot example contexts were developed as described in Section 3.3, while semantically relevant relationships between tables and columns, along with likely join paths between the retrieved nodes, were derived from a knowledge graph constructed over the database schema, as detailed in Section 3.4. An example of the extracted triples used to construct the knowledge graph is presented in Table 3, and a simplified visualization of the resulting knowledge graph is shown in Fig. 5. Contexts retrieved from both systems were integrated into the prompt provided to the fine-tuned language models.

To optimize overall system performance, we first tuned the retrieval depth of the RAG component by varying the Top_k parameter, which controls the number of retrieved textual documents. The impact of different Top_k values was evaluated using execution accuracy and F1 score, with results summarized in Table 1.

For the knowledge graph component, we conducted a systematic sensitivity analysis, examining the effect of varying the number of retrieved knowledge graph entries (k_{kg}) and the number of candidate join paths (k_{join}) on SQL generation performance. Each configuration was evaluated over five independent runs, and results are reported as mean ± standard deviation in Table 2.

The analysis shows that performance improves as k_{kg} and k_{join} increase from small values, peaking at $k_{\text{kg}} = 3$ and $k_{\text{join}} = 3$. This configuration achieves the highest mean execution accuracy (94.75%) and F1 score, while also exhibiting relatively low variance across runs, indicating stable behavior. Further increasing either parameter does not yield consistent performance gains and, in some cases, slightly degrades performance due to the introduction of less relevant or noisy knowledge graph entries.

These results demonstrate that the chosen configuration represents a robust empirical trade-off between providing sufficient structural context for SQL generation and maintaining resistance to noise, thereby validating the proposed schema-guided RAG framework's sensitivity to knowledge retrieval depth.

The open-source model was fine-tuned using the Low-Rank Adaptation (LoRA) method, as outlined in Section 3.2. Given the limited size of the domain-specific SQL-natural language dataset used in this study, a low-rank configuration was adopted to mitigate overfitting while allowing the model to learn meaningful structural and semantic patterns. To identify an optimal configuration, a hyperparameter search was performed using Optuna, proposed by Akiba et al. [47], with a median pruning strategy over 20 trials. The progression of evaluation loss across trials is shown in Fig. 6, and the best-performing hyperparameter values are summarized in Table 5. Despite the small scale of fine-tuning, this approach yielded improved alignment between query intent and SQL structure, particularly when combined with schema-guided RAG-enhanced prompts.

As illustrated in Fig. 4, the schema-guided RAG-enhanced prompt is passed to the fine-tuned model to generate the SQL code required

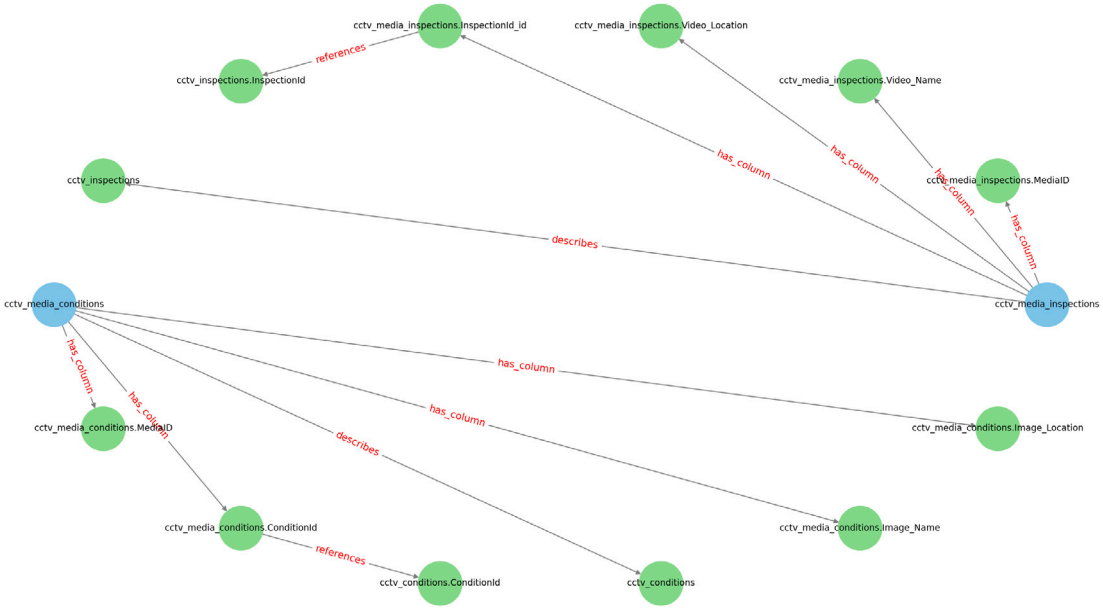


Fig. 5. Knowledge graph example constructed from two tables in the relational database.

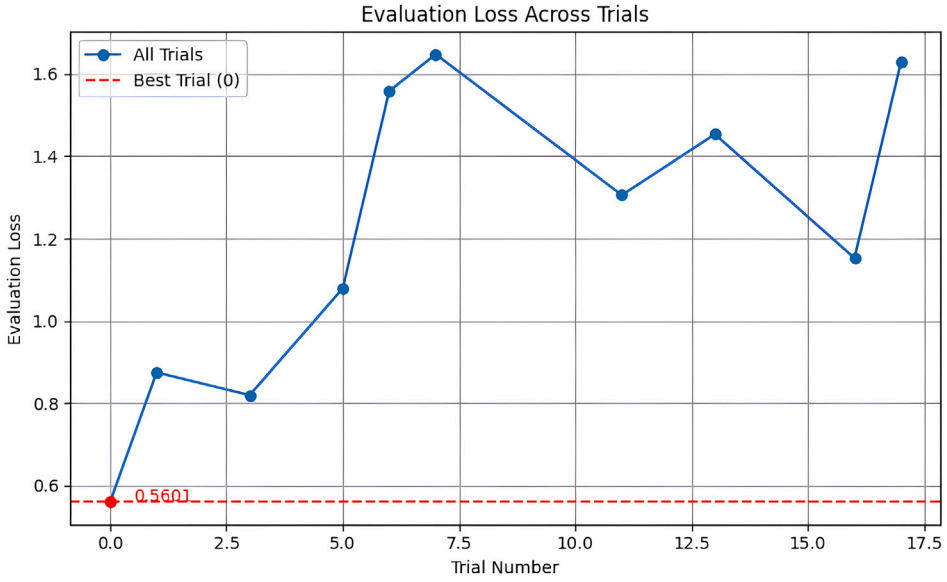


Fig. 6. Evaluation loss across Optuna hyperparameter optimization trials. The best trial (Trial 0) achieved a loss of 0.5601. Later trials did not improve performance, likely due to suboptimal parameter sampling or overfitting.

to query the database. The generated SQL queries are then executed against the sewer asset management database, and the retrieved results are compared with the expected outputs to assess both the execution correctness and semantic alignment.

This framework illustrates a comprehensive approach to integrating structured domain knowledge, specifically through knowledge graph-enhanced context, with retrieval-augmented generation and fine-tuned language models to tackle sophisticated SQL query generation tasks in real-world, domain-specific data bases.

4.3. Ablation studies

To assess the effectiveness of the schema-guided RAG architecture and its components, we conducted an ablation study across three large language models: GPT-4o-mini, GPT-3.5 Turbo (both from OpenAI), and the open-source SQL-Coder from Defog, Inc. Each model was evaluated under four incremental configurations: (1) the base model with no finetuning or retrieval, (2) with finetuning only, (3) with finetuning and RAG-based metadata retrieval, and (4) with the full schema-guided RAG system, incorporating the knowledge graph.

As shown in Table 4, each component (fine-tuning, retrieval-augmented generation (RAG), and knowledge graph (KG)) incrementally improved performance across execution accuracy, logical form accuracy, and exact match metrics. GPT-4o-mini demonstrated the most significant gains with the full schema-guided RAG configuration. Starting from a base execution accuracy of just 5.97%, fine-tuning alone boosted this to 50.75%, while the addition of RAG increased it further to 87.50%. The full schema-guided RAG system achieved a peak execution accuracy of 95.31%, LF accuracy of 85.94%, and exact match rate of 94.69%. GPT-3.5 Turbo followed a similar trajectory, with execution accuracy advancing from 5.97% to 81.25%, logical form accuracy from

Table 2
Retrieved entries (k_{kg}) and join paths (k_{join}) sensitivity analysis for knowledge graph. Results are reported as mean ± standard deviation over five runs.

k_{kg}	k_{join}	Execution accuracy (%)	F1 score (%)
0	0	86.62 ± 0.00	86.52 ± 0.00
0	1	87.31 ± 1.31	87.16 ± 1.31
0	3	87.56 ± 1.40	89.35 ± 1.59
0	5	88.81 ± 0.86	90.91 ± 0.70
0	7	88.81 ± 0.86	90.50 ± 0.77
0	10	87.56 ± 0.86	89.98 ± 0.86
1	0	87.75 ± 0.00	87.75 ± 0.00
1	1	87.56 ± 1.40	88.41 ± 1.40
1	3	89.38 ± 0.86	90.22 ± 0.86
1	5	90.31 ± 1.10	91.16 ± 1.10
1	7	91.69 ± 0.86	92.53 ± 0.86
1	10	92.00 ± 1.31	92.95 ± 1.36
3	0	93.75 ± 0.00	94.27 ± 0.00
3	1	94.06 ± 0.70	94.91 ± 0.70
3	**3**	**94.38 ± 1.40**	**95.22 ± 1.40**
3	5	93.62 ± 0.70	94.47 ± 0.70
3	7	92.00 ± 0.70	92.85 ± 0.70
3	10	92.00 ± 0.70	92.85 ± 0.70
5	0	90.00 ± 0.70	90.21 ± 0.93
5	1	87.44 ± 1.31	89.39 ± 1.20
5	3	89.38 ± 0.86	90.22 ± 0.86
5	5	89.38 ± 1.78	90.22 ± 1.78
5	7	90.62 ± 1.31	91.47 ± 1.31
5	10	90.62 ± 0.70	91.47 ± 0.70
7	0	91.25 ± 0.86	91.90 ± 0.29
7	1	89.38 ± 1.78	90.22 ± 1.78
7	3	92.19 ± 1.31	93.14 ± 1.30
7	5	90.00 ± 0.70	91.06 ± 0.57
7	7	89.69 ± 0.86	90.64 ± 0.97
7	10	90.62 ± 1.31	91.78 ± 1.07
10	0	89.44 ± 1.31	89.49 ± 1.07
10	1	89.69 ± 1.40	90.34 ± 1.30
10	3	92.81 ± 1.40	92.98 ± 1.30
10	5	92.81 ± 0.86	92.42 ± 0.56
10	7	93.12 ± 1.71	93.29 ± 1.59
10	10	92.19 ± 1.31	92.70 ± 0.93

Table 3
Example of triples used for building the knowledge graph.

Subject	Predicate	Object
`cctv_media_conditions c`	has_column	`c.Image_Location`
`cctv_media_inspections i`	Describes	`cctv_inspections`
`cctv_inspections.Inspectionid`	References	`i.Inspectionid_id`

Table 4
Ablation study across three large language models.

Model	Variant	FT	RAG	KG	Exec. Acc.	LF Acc.	Exact match
GPT-4o-mini	Base	✗	✗	✗	5.97%	0.00%	27.76%
	+ Finetuned	✓	✗	✗	50.75%	23.88%	64.18%
	+ RAG	✓	✓	✗	87.50%	81.25%	92.50%
	+ KG	✓	✓	✓	**95.31%**	**85.94%**	**94.69%**
GPT-3.5 Turbo	Base	✗	✗	✗	5.97%	0.00%	26.27%
	+ Finetuned	✓	✗	✗	49.25%	28.36%	66.27%
	+ RAG	✓	✓	✗	71.88%	50.00%	80.00%
	+ KG	✓	✓	✓	81.25%	67.19%	86.56%
SQL-Coder	Base	✗	✗	✗	0.00%	0.00%	25.00%
	+ Finetuned	✓	✗	✗	28.12%	12.50%	50.00%
	+ RAG	✓	✓	✗	67.19%	50.00%	75.94%
	+ KG	✓	✓	✓	73.75%	53.44%	80.31%

0.00% to 67.19%, and exact match from 26.27% to 86.56%. SQL-Coder, despite a baseline of 0% execution and logical form accuracy, achieved 73.75% execution accuracy, 53.44% logical form accuracy, and 80.31% exact match with the full schema-guided RAG configuration. The consistent performance improvements across all models underscore the transferability of the schema-guided RAG approach. Ablation of any component (fine-tuning, RAG, or KG) resulted in substantial performance declines, with the knowledge graph providing the critical final enhancement, particularly for modeling relational structures and join paths in domain-specific SQL query generation.

These results validate the effectiveness and adaptability of the schema-guided RAG framework across both proprietary and open-source models.

Table 5
Optimal hyperparameters identified by Optuna during fine-tuning.

Hyperparameter	Optimal value
Learning rate	6.13×10^{-5}
Batch size	4
LoRA rank (r)	31
LoRA alpha	63
LoRA dropout	0.0683

5. Discussion

This study proposes a hybrid framework that integrates structured domain knowledge via knowledge graphs (KGs) and contextual retrieval through retrieval-augmented generation (RAG) to enhance the performance of large language models (LLMs) in text-to-SQL generation for specialized databases. The framework consists of a three-component pipeline (KG construction, semantic retrieval, and knowledge-enhanced prompting) that addresses core limitations in conventional LLM-based semantic parsing.

The knowledge graph captures schema-specific semantics, including foreign key relationships, hierarchical entity types, and relational join paths. This schema layer enables LLMs to reason over complex relational structures, improving the generation of syntactically correct and executable SQL queries, particularly in multi-table databases. Across all model configurations, the KG consistently improved performance, especially for queries involving nested subqueries or implicit joins.

The RAG component injects relevant schema descriptions and example queries into prompts, thereby enhancing in-context learning. By retrieving relational knowledge from unstructured schema documentation, RAG improves the LLM's ability to generalize to unseen query intents. This is particularly effective in domain-specific settings where labeled training data is limited.

The framework was evaluated on both proprietary (GPT-4o-mini, GPT-3.5 Turbo) and open-source (SQL-Coder) LLMs, demonstrating consistent improvements in execution accuracy, logical form accuracy, and exact match rates. These results confirm the generalizability of the schema-guided RAG approach. Notably, even the resource-constrained SQL-Coder model achieved substantial relative gains, highlighting the framework's adaptability in low-resource settings.

The schema-guided RAG framework is particularly well-suited for the sewer asset management domain, where it has been deployed. Sewer infrastructure databases often contain complex, interrelated tables covering pipe networks, inspection logs, and maintenance records. In this context, the framework enables accurate and interpretable SQL generation, empowering engineers and asset managers to retrieve actionable insights, such as identifying pipes that require urgent maintenance without requiring SQL expertise.

While the experimental scope in this study was limited to sewer asset management, the underlying schema-guided RAG methodology is domain-agnostic. Its schema encoding, retrieval mechanism, and prompting strategies can be readily applied to other relational domains, such as transportation, energy, or healthcare, with minimal adaptation. This highlights the potential for broader applicability beyond the current domain.

Despite its demonstrated effectiveness, the framework has several limitations. First, it assumes a static database schema, which limits its applicability in dynamic or evolving environments. Future work should explore methods for incremental KG construction and updates. Second, the system does not explicitly model user intent or query context, which could be addressed through session-based modeling or interactive clarification. Finally, the current design is limited to SQL-based systems; extending it to alternative query languages such as NoSQL and Datalog is a promising direction for expanding its reach across data platforms.

The schema-guided RAG framework offers an interpretable and extensible solution for text-to-SQL generation in complex, domain-specific databases. By combining structured schema representation, contextual retrieval, and LLM reasoning, it enhances query accuracy and generalizability, facilitating reliable and efficient access to critical data in sewer asset management and other specialized domains.

6. Conclusion

This study introduces schema-guided RAG, a hybrid framework that integrates structured schema representation through knowledge graphs (KGs) and contextual retrieval via retrieval-augmented generation (RAG) to enhance text-to-SQL generation for complex, domain-specific relational databases. By combining schema knowledge with large language model (LLM) reasoning, schema-guided RAG addresses challenges in semantic parsing, including accurate join reasoning, limited training data, and interpretability.

Experimental results across both proprietary (GPT-4o-mini, GPT-3.5 Turbo) and open-source (SQL-Coder) LLMs demonstrate consistent improvements in execution accuracy, logical form accuracy, and exact match rates, confirming the framework's generalizability across model scales. The KG component significantly enhances performance in complex queries involving multi-table joins and nested structures. Tested in the sewer asset management domain, schema-guided RAG enables non-SQL -expert users to generate precise, executable SQL queries over intricate infrastructure datasets, supporting critical tasks such as maintenance prioritization and inspection tracking.

While promising, the framework has limitations, including its assumption of a static schema and the lack of explicit modeling of user intent. Future work should explore dynamic KG updating, integration of user interaction, and adaptation to other data modalities and query languages.

In conclusion, schema-guided RAG provides an interpretable and extensible solution for domain-specific text-to-SQL generation, offering a scalable foundation for intelligent database interfaces in sewer asset management and related fields.

CRediT authorship contribution statement

Ifeoluwa Awotunde: Writing – original draft, Methodology. **Dharmendra Reddy Chitte:** Software. **Yongwei Shan:** Writing – review & editing, Funding acquisition, Conceptualization. **Weihua Sheng:** Writing – review & editing, Funding acquisition, Conceptualization. **Hossein Khaleghian:** Supervision, Funding acquisition, Data curation.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the authors used ChatGPT (OpenAI, San Francisco, CA) to enhance readability and improve the clarity of the manuscript. After using this tool, the authors reviewed and edited the content as necessary, taking full responsibility for the accuracy and content of the publication.

Funding

This research was funded by the Oklahoma Center for the Advancement of Science & Technology (OCAST) (grant no. AR24-003).

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

[1] American Society of Civil Engineers, 2025 Infrastructure Report Card: Wastewater, 2025, https://www.infrastructurereportcard.org/cat-item/wastewater/.
[2] U.S. Environmental Protection Agency, Report to Congress: Impacts and Control of CSOs and SSOs, 2004, EPA 833-R-04-001.
[3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, Adv. Neural Inf. Process. Syst. 30 (2017).
[4] C. Baik, H.V. Jagadish, Y. Li, Bridging the semantic gap with SQL query logs in natural language interfaces to databases, in: 2019 IEEE 35th International Conference on Data Engineering, ICDE, IEEE, 2019, pp. 374–385.
[5] F. Li, H.V. Jagadish, Constructing an interactive natural language interface for relational databases, Proc. VLDB Endow. 8 (1) (2014) 73–84.
[6] J. Sen, C. Lei, A. Quamar, F. Özcan, V. Efthymiou, A. Dalmia, G. Stager, A. Mittal, D. Saha, K. Sankaranarayanan, Athena++ natural language querying for complex nested sql queries, Proc. VLDB Endow. 13 (12) (2020) 2747–2759.
[7] T. Mahmud, K.A. Hasan, M. Ahmed, T.H.C. Chak, A rule based approach for NLP based query processing, in: 2015 2nd International Conference on Electrical Information and Communication Technologies, EICT, IEEE, 2015, pp. 78–82.
[8] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Comput. 9 (8) (1997) 1735–1780.
[9] J. Chung, C. Gulcehre, K. Cho, Y. Bengio, Empirical evaluation of gated recurrent neural networks on sequence modeling, 2014, arXiv preprint arXiv:1412.3555.
[10] V. Zhong, C. Xiong, R. Socher, Seq2sql: Generating structured queries from natural language using reinforcement learning, 2017, arXiv preprint arXiv:1709.00103.
[11] X. Xu, C. Liu, D. Song, Sqlnet: Generating structured queries from natural language without reinforcement learning. arxiv 2017, 2017, arXiv preprint arXiv:1711.04436.
[12] B. Wang, R. Shin, X. Liu, O. Polozov, M. Richardson, Rat-sql: Relation-aware schema encoding and linking for text-to-sql parsers, 2019, arXiv preprint arXiv:1911.04942.
[13] T. Scholak, N. Schucher, D. Bahdanau, PICARD: Parsing incrementally for constrained auto-regressive decoding from language models, 2021, arXiv preprint arXiv:2109.05093.
[14] B. Min, H. Ross, E. Sulem, A.P.B. Veyseh, T.H. Nguyen, O. Sainz, E. Agirre, I. Heintz, D. Roth, Recent advances in natural language processing via large pre-trained language models: A survey, ACM Comput. Surv. 56 (2) (2023) 1–40.
[15] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F.L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, et al., Gpt-4 technical report, 2023, arXiv preprint arXiv:2303.08774.
[16] M. Chen, J. Tworek, H. Jun, Q. Yuan, H.P.D.O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al., Evaluating large language models trained on code, 2021, arXiv preprint arXiv:2107.03374.
[17] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al., Llama: Open and efficient foundation language models, 2023, arXiv preprint arXiv:2302.13971.
[18] M. Pourreza, D. Rafiei, Din-sql: Decomposed in-context learning of text-to-sql with self-correction, Adv. Neural Inf. Process. Syst. 36 (2023) 36339–36348.
[19] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, J. Zhou, Text-to-sql empowered by large language models: A benchmark evaluation, 2023, arXiv preprint arXiv:2308.15363.
[20] T. Ren, Y. Fan, Z. He, R. Huang, J. Dai, C. Huang, Y. Jing, K. Zhang, Y. Yang, X.S. Wang, Purple: Making a large language model a better sql writer, in: 2024 IEEE 40th International Conference on Data Engineering, ICDE, IEEE, 2024, pp. 15–28.
[21] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive NLP tasks, 2021, arXiv:2005.11401 URL https://arxiv.org/abs/2005.11401.
[22] H. Koziolek, S. Grüner, R. Hark, V. Ashiwal, S. Linsbauer, N. Eskandani, LLM-based and retrieval-augmented control code generation, in: Proceedings of the 1st International Workshop on Large Language Models for Code, 2024, pp. 22–29.
[23] J.A.H. Álvaro, J.G. Barreda, An advanced retrieval-augmented generation system for manufacturing quality control, Adv. Eng. Inform. 64 (2025) 103007.

[24] H. Park, M. Rees, N. Kruger, K. Fuse, V.M. Castro, V. Gainer, N. Wattanasin, B. Benoit, K.B. Wagholikar, S.N. Murphy, A comprehensive evaluation of LLM phenotyping using retrieval-augmented generation (RAG): Insights for RAG optimization, 2025, MedRxiv, 2025-2004.

[25] Y. Xie, B. Jiang, T. Mallick, J.D. Bergerson, J.K. Hutchison, D.R. Verner, J. Branham, M.R. Alexander, R.B. Ross, Y. Feng, et al., A rag-based multi-agent llm system for natural hazard resilience and adaptation, 2025, arXiv preprint arXiv:2504.17200.

[26] C. Gondhalekar, U. Patel, F.-C. Yeh, MultiFinRAG: An optimized multi-modal retrieval-augmented generation (RAG) framework for financial question answering, 2025, arXiv preprint arXiv:2506.20821.

[27] W. Xiao, D. Ekberg, S. Garg, R. Karri, Hybrid-NL2SVA: Integrating RAG and finetuning for LLM-based NL2SVA, 2025, arXiv preprint arXiv:2506.21569.

[28] S. Gopi, D. Sreekanth, N. Dehbozorgi, Enhancing engineering education through LLM-driven adaptive quiz generation: A RAG-based approach, in: 2024 IEEE Frontiers in Education Conference, FIE, IEEE, 2024, pp. 1–8.

[29] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R.O. Ness, J. Larson, From local to global: A graph rag approach to query-focused summarization, 2024, arXiv preprint arXiv:2404.16130.

[30] Y. Yang, H. Wu, T. Wang, J. Yang, H. Ma, G. Luo, Pseudo-knowledge graph: Meta-path guided retrieval and in-graph text for RAG-equipped LLM, 2025, arXiv preprint arXiv:2503.00309.

[31] D. Li, Z. Li, Y. Yang, L. Sun, D. An, Q. Yang, Knowledge graph-enhanced large language model for domain-specific question answering systems, 2024, Authorea Preprints.

[32] C. Zheng, S. Wong, X. Su, Y. Tang, A. Nawaz, M. Kassem, Automating construction contract review using knowledge graph-enhanced large language models, Autom. Constr. 175 (2025) 106179.

[33] J. Guo, Y. Wang, S. Li, G. Chen, A knowledge graph-based standardized modeling and quantitative retrieval method for product defect analysis-related knowledge, Adv. Eng. Inform. 66 (2025) 103372.

[34] C. Ma, S. Chakrabarti, A. Khan, B. Molnár, Knowledge graph-based retrieval-augmented generation for schema matching, 2025, arXiv preprint arXiv:2501.08686.

[35] A. Ziletti, L. D'Ambrosi, Generating patient cohorts from electronic health records using two-step retrieval-augmented text-to-SQL generation, 2025, arXiv preprint arXiv:2502.21107.

[36] E.J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, et al., Lora: Low-rank adaptation of large language models., ICLR 1 (2) (2022) 3.

[37] T. Dettmers, A. Pagnoni, A. Holtzman, L. Zettlemoyer, Qlora: Efficient finetuning of quantized llms, 2023 2 (2023) URL https://arxiv.Org/Abs/2305.14314.

[38] X. Liu, K. Ji, Y. Fu, W.L. Tam, Z. Du, Z. Yang, J. Tang, P-tuning v2: Prompt tuning can be comparable to fine-tuning universally across scales and tasks, 2021, arXiv preprint arXiv:2110.07602.

[39] OpenAI, Fine-tuning GPT models, 2023, (Accessed: 15 July 2025) https://platform.openai.com/docs/guides/fine-tuning.

[40] O. Ram, Y. Levine, I. Dalmedigos, D. Muhlgay, A. Shashua, K. Leyton-Brown, Y. Shoham, In-context retrieval-augmented language models, Trans. Assoc. Comput. Linguist. 11 (2023) 1316–1331.

[41] X. Xie, H. Liu, W. Hou, H. Huang, A brief survey of vector databases, in: 2023 9th International Conference on Big Data and Information Analytics, BigDIA, IEEE, 2023, pp. 364–371.

[42] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, H. Wang, H. Wang, Retrieval-augmented generation for large language models: A survey 2 (1) (2023) arXiv preprint arXiv:2312.10997.

[43] Google, Introducing the knowledge graph: things, not strings, 2012, (Accessed: 16 July 2025) https://blog.google/products/search/introducing-knowledge-graph-things-not/.

[44] F. Shi, L. Chen, M. Zhou, Y. Zhao, Y. Zheng, A stepwise intelligence generative method for structured maintenance guidance documents based on knowledge graph augmented LLM, Adv. Eng. Inform. 67 (2025) 103523.

[45] D. Zhang, G. Ma, T. Qu, X. Wang, W. Zhou, X. Wang, A knowledge graph-enhanced large language model for question answering of hydraulic structure safety management, Adv. Eng. Inform. 66 (2025) 103468.

[46] A.B. Kanburoğlu, F.B. Tek, Text-to-SQL: A methodical review of challenges and models, Turk. J. Electr. Eng. Comput. Sci. 32 (3) (2024) 403–419.

[47] T. Akiba, S. Sano, T. Yanase, T. Ohta, M. Koyama, Optuna: A next-generation hyperparameter optimization framework, in: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2019, pp. 2623–2631.